

ПРАКТИЧНА РОБОТА № 3

СТРУКТУРНІ ШАБЛОНИ ПРОЕКТУВАННЯ ПЗ. COMPOSITE, DECORATOR, PROXY, FLYWEIGHT, ADAPTER, BRIDGE, FACADE.

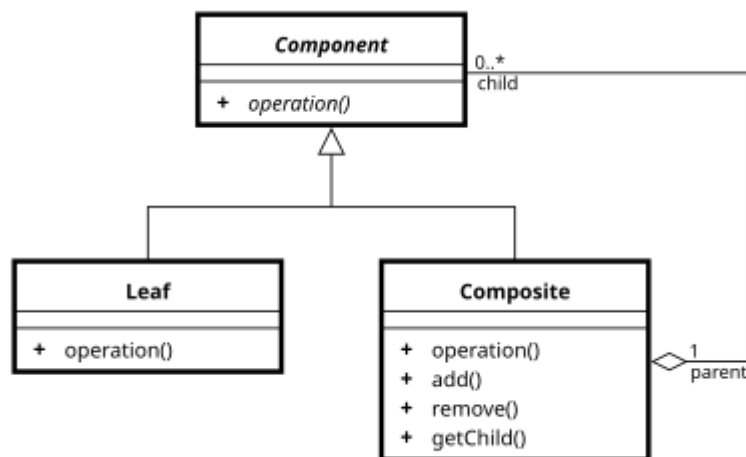
Мета: Ознайомлення з видами шаблонів проектування ПЗ. Вивчення структурних шаблонів. Отримання базових навичок з застосування шаблонів Composite, Decorator, Proxy, Flyweight, Adapter, Bridge, Facade.

Короткі теоретичні відомості

Шаблон Composite (Компонувальник)

Проблема. Як зробити так, щоб працювати з окремими об'єктами та групами об'єктів однаковою способом?

Рішення. Створюємо класи для окремих об'єктів (їх називають "листами") та для груп об'єктів (їх називають "композиціями"). Ці класи реалізують один і той самий інтерфейс, тому ми можемо обробляти і окремі об'єкти, і їхні групи однаково.



Мал. 1. Структура шаблону Composite

Учасники шаблону:

1. Component (Компонент).

- Це спільний інтерфейс, який використовують і окремі об'єкти, і групи об'єктів.
- Він визначає загальні методи для роботи з об'єктами.
- Також може мати методи для додавання/видалення підлеглих об'єктів, якщо це потрібно.

2. Leaf (Листок).

- Це окремий елементарний об'єкт, який не має підлеглих.
- Він реалізує поведінку для окремих, простих об'єктів у системі.

3. Composite (Складений об'єкт).

- Це складений об'єкт, який містить інші об'єкти (як листки, так і інші композиції).
- Він реалізує поведінку для групи об'єктів.
- Може додавати, видаляти та керувати підлеглими через інтерфейс Component.

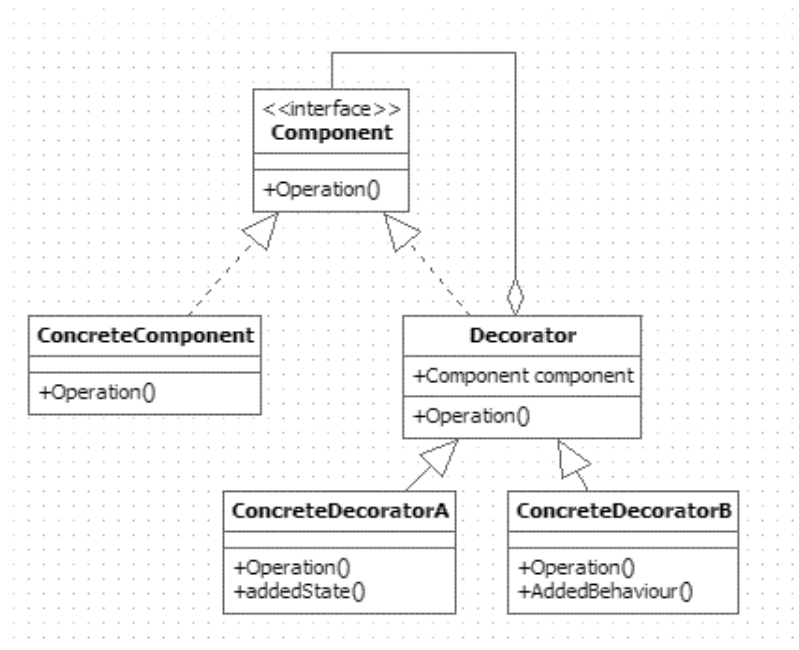
4. Client (Клієнт).

- Це код, який працює з об'єктами через інтерфейс Component.
- Для клієнта не має значення, чи це окремий об'єкт (листок) або група об'єктів (композит), оскільки він використовує один і той самий інтерфейс.

Шаблон Decorator

Проблема. Як додати нові обов'язки або функціонал до окремого об'єкта, не змінюючи весь клас?

Рішення. В динаміці додати об'єкту нові обов'язки, не вдаючись при цьому до породження підкласів (наслідування). "Component" визначає інтерфейс для об'єктів, на які можуть бути в динаміці покладені додаткові обов'язки, "ConcreteComponent" визначає об'єкт, на який покладаються додаткові обов'язки, "Decorator" – зберігає посилання на об'єкт "Component" і визначає інтерфейс, відповідний інтерфейсу "Component". "ConcreteDecorator" покладає додаткові обов'язки на компонент. "Decorator" переадресує запити об'єкту "Component".



Мал. 2. Структура шаблону Decorator

Учасники шаблону:

1. **Component.** Визначає інтерфейс для об'єктів, до яких можна динамічно додавати нові обов'язки.
2. **ConcreteComponent** – це реальний об'єкт, який реалізує інтерфейс Component і може отримувати додаткові обов'язки.
3. **Decorator** – це клас, який зберігає посилання на об'єкт Component і реалізує той самий інтерфейс. Він діє як обгортка, що може змінювати поведінку компонента.
4. **ConcreteDecorator.** Реалізує конкретний декоратор, який додає нові обов'язки або функціональність до об'єкта Component.

Застосування декількох декораторів до одного компонента дозволяє комбінувати різні функції в довільному порядку. Наприклад, можна додати одну й ту ж саму властивість двічі, що надає гнучкість. На відміну від статичного наслідування, де для кожного нового обов'язку потрібно створювати новий клас, декоратори дозволяють додавати і видаляти обов'язки під час виконання програми. Це допомагає уникнути великих класів з багатьма методами у верхній частині ієрархії — замість цього, обов'язки додаються за потреби.

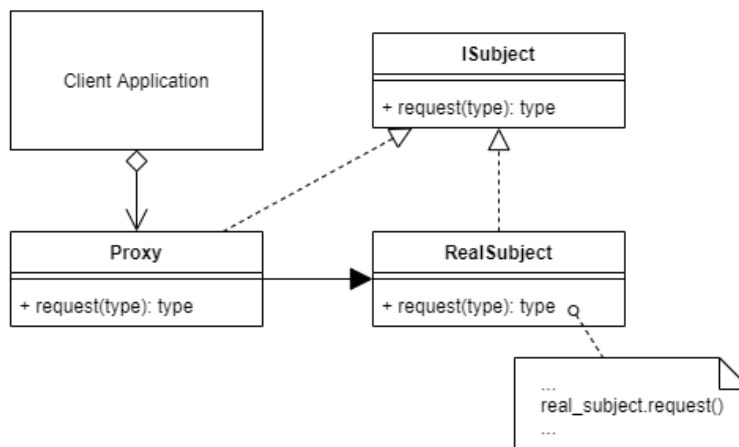
Проте декоратори і компоненти, на які вони застосовуються, не є повністю однаковими. В результаті, система може складатися з великої кількості дрібних об'єктів, які дуже схожі між собою і відрізняються тільки тим, як вони взаємодіють. Це може ускладнити розуміння і налагодження системи, оскільки вона стає важчою для аналізу.

Шаблон Proxy

Проблема. Потрібно керувати доступом до об'єкта для таких цілей:

- створення громіздких об'єктів лише тоді, коли вони потрібні (Virtual Proxy);
- кешування даних (SmartLink Proxy);
- обмеження доступу в реальному часі (Security Proxy);
- забезпечення доступу до об'єкта, що знаходиться в іншому адресному просторі (Remote Proxy).

Рішення. Створюємо **сурогат** для складного об'єкта. Цей сурогат називається **Proxy** і зберігає посилання на реальний об'єкт (**RealSubject**). Proxy контролює доступ до цього об'єкта, створює або видаляє його, коли це потрібно. Загальний інтерфейс для Proxy і RealSubject визначається в класі **Subject**, щоб Proxy можна було використовувати там, де очікується RealSubject. Proxy може переадресовувати запити до реального об'єкта, коли це потрібно.



Мал. 3. Структура шаблону Proxy

Учасники шаблону:

1. Proxy (ImageProxy).

- Зберігає посилання на реальний об'єкт (RealSubject), до якого можна отримати доступ.
- Має той самий інтерфейс, що й реальний об'єкт, тому може замінити його.
- Контролює доступ до реального об'єкта, відповідає за його створення або видалення.
- В залежності від типу Proxy, він може:
 - Віддалено пересилати запити реальному об'єкту в іншому адресному просторі (Remote Proxy).
 - Кешувати дані, щоб відкласти створення реального об'єкта (Virtual Proxy).
 - Перевіряти права доступу користувача до реального об'єкта (Security Proxy).

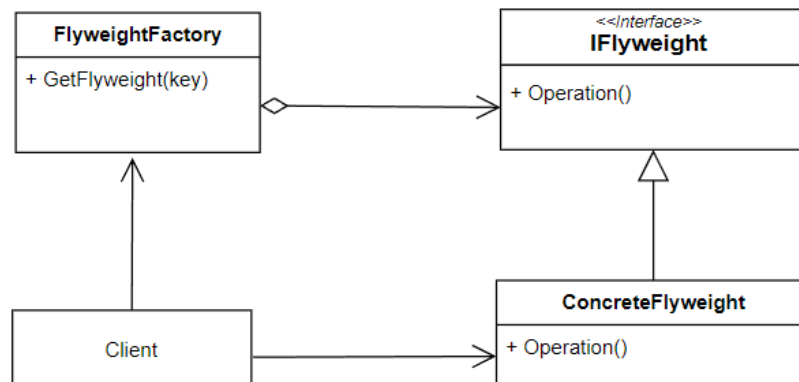
2. **Subject (Graphic).** Визначає загальний інтерфейс для реального об'єкта (RealSubject) і його заступника (Proxy), щоб Proxy міг використовуватися замість реального об'єкта в будь-якому місці.
3. **RealSubject (Image).** Це реальний об'єкт, до якого звертається Proxy, і який виконує реальні дії, що представляє заступник.

Шаблон Flyweight (Легковаговик)

Проблема. Необхідно забезпечити підтримку великої кількості дрібних об'єктів.

Рішення. Створити об'єкт, який можна використовувати одночасно в кількох контекстах, причому, в кожному контексті об'єкт виглядає як незалежний об'єкт (не відрізняється від екземпляра, який не розділяється). "Flyweight" оголошує інтерфейс, за допомогою якого пристосуванці можуть отримати зовнішній стан або якось впливати на нього, "ConcreteFlyweight" реалізує інтерфейс класу "Flyweight" і додає за необхідності внутрішній стан. Внутрішній стан зберігається в об'єкті "ConcreteFlyweight", в той час як зовнішній стан зберігається або обчислюється в "Client" ("Client" передає його "Flyweight" при виклику операцій).

Об'єкт класу "ConcreteFlyweight" розділяється. Будь-який стан у ньому має бути внутрішнім, тобто незалежним від контексту, "FlyweightFactory" - створює об'єкти - "Flyweight" (або надає примірник, що вже існує) та керує ними. "UnsharedConcreteFlyweight" - не всі підкласи "Flyweight" обов'язково розділяються. "Client" - зберігає посилання на одного або кількох "Flyweight", обчислює і зберігає зовнішній стан "Flyweight".



Мал. 1. Структура шаблону Flyweight

Учасники шаблону:

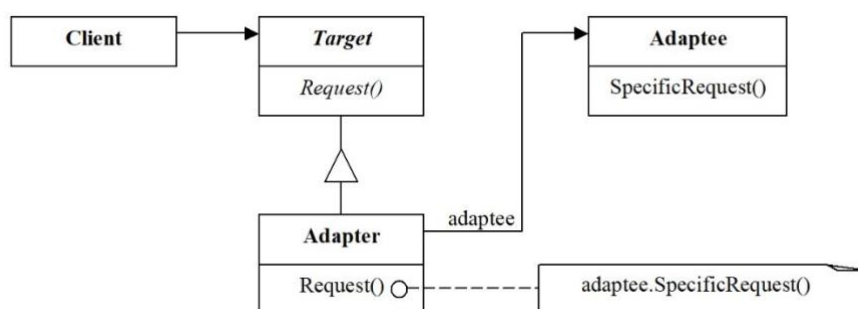
1. **Flyweight** – легковаговик, цей компонент призначений для оголошення інтерфейсу, за допомогою якого пристосуванці можуть отримувати зовнішній стан і якось змінювати його.
2. **ConcreteFlyweight** – конкретний легковаговик, цей компонент призначений для реалізації інтерфейсу класа Flyweight та додавання, в разі потреби, внутрішнього стану. Об'єкт класу ConcreteFlyweight повинен бути розподіленим (тобто спільним). Будь-який стан, який він зберігає, повинен бути внутрішнім, тобто він повинен бути незалежним від контексту.
3. **UnsharedConcreteFlyweight** – неподільний конкретний легковаговик, цей компонент призначений для закріплення певних підкласів Flyweight, тобто не всі підкласи мають бути поділені. Інтерфейс Flyweight дозволяє поділ, але не нав'язує його. Часто об'єкти UnsharedConcreteFlyweight мають об'єкти ConcreteFlyweight в якості нащадків на якомусь рівні структури пристосування.
4. **FlyweightFactory** – фабрика легковаговиків, цей компонент призначений для створення та управління об'єктами Flyweight, а також для забезпечення належного поділу легковаговиків. Якщо клієнт запитує легковаговика, об'єкт FlyweightFactory надає існуючий екземпляр або створює новий екземпляр, якщо такий не існує.

5. **Client** – клієнт, цей компонент призначений для зберігання посилання на легковаговика(ів) та для обчислення або зберігання зовнішнього стану легковаговика(ів).

Шаблон Adapter

Проблема. Необхідно забезпечити взаємодію несумісних інтерфейсів або як створити єдиний стійкий інтерфейс для декількох компонентів з різними інтерфейсами.

Рішення. Конвертувати вихідний інтерфейс компонента до іншого виду за допомогою проміжного об'єкта - адаптера, тобто, додати спеціальний об'єкт із загальним інтерфейсом в рамках даної програми і перенаправити зв'язок від зовнішніх об'єктів до цього об'єкта - адаптера. Структура шаблону наведена на мал. 2.



Мал. 2. Структура шаблону Adapter

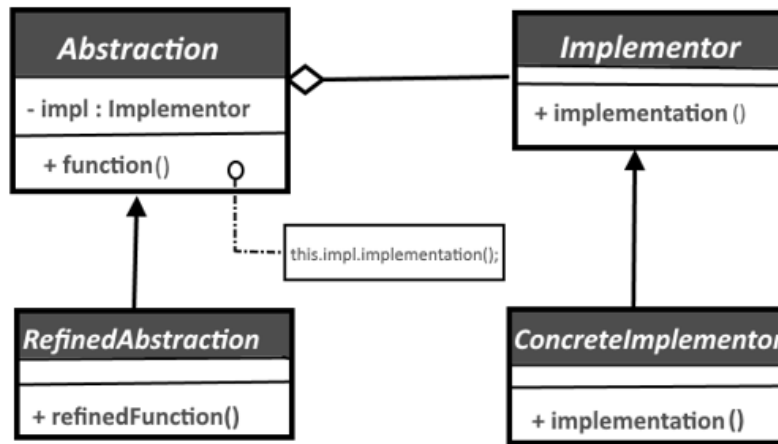
Учасники шаблону:

1. **Цільовий (Target) інтерфейс** – це інтерфейс, який очікує клієнтський код. Він представляє бажаний інтерфейс, з яким клієнт хоче працювати.
2. **Адаптований (Adaptee) клас** – це клас або інтерфейс, який несумісний з цільовим інтерфейсом. Цей клас потрібно адаптувати, щоб він міг бути використаний клієнтом.
3. **Адаптер (Adapter)** – це клас, який реалізує цільовий інтерфейс та внутрішньо містить об'єкт адаптованого класу. Адаптер перетворює виклики методів цільового інтерфейсу в виклики методів адаптованого класу.

Шаблон Bridge

Проблема. Потрібно відділити абстракцію від реалізації так, щоб і те, і інше можна було змінювати незалежно.

Рішення. Помістити абстракцію та реалізацію в окремі ієрархії класів. "Abstraction" визначає інтерфейс "Abstraction" і зберігає посилання на об'єкт "Implementor", "RefinedAbstraction" розширює інтерфейс, заданий у "Abstraction". "Implementor" визначає інтерфейс для класів реалізації, він не зобов'язаний точно відповідати інтерфейсу класу "Abstraction" - обидва інтерфейси можуть бути зовсім різні. Зазвичай інтерфейс класу "Implementor" надає тільки примітивні операції, а клас "Abstraction" визначає операції більш високого рівня, що базуються на цих примітивних. "ConcretelImplementor" містить конкретну реалізацію класу "Implementor". Об'єкт "Abstraction" перенаправляє запити "Client" до свого об'єкту "Implementor". Структура шаблону наведена на мал.3.



Мал. 3. Структура шаблону Bridge

Учасники шаблону:

4. Абстракція (Abstraction) – це інтерфейс або абстрактний клас, який визначає вищий рівень операцій або функціональність та містить посилання на об'єкт реалізації.

5. Реалізація (Implementation) – це інтерфейс або абстрактний клас, який визначає нижчий рівень конкретної реалізації операцій абстракції.

6. Конкретна абстракція (Concrete Abstraction) – реалізація абстракції, яка спадкується від абстракції і використовує об'єкт реалізації для виконання своїх операцій.

7. Конкретна реалізація (Concrete Implementation) – реалізація реалізації, яка спадкується від реалізації і надає конкретну реалізацію операцій.

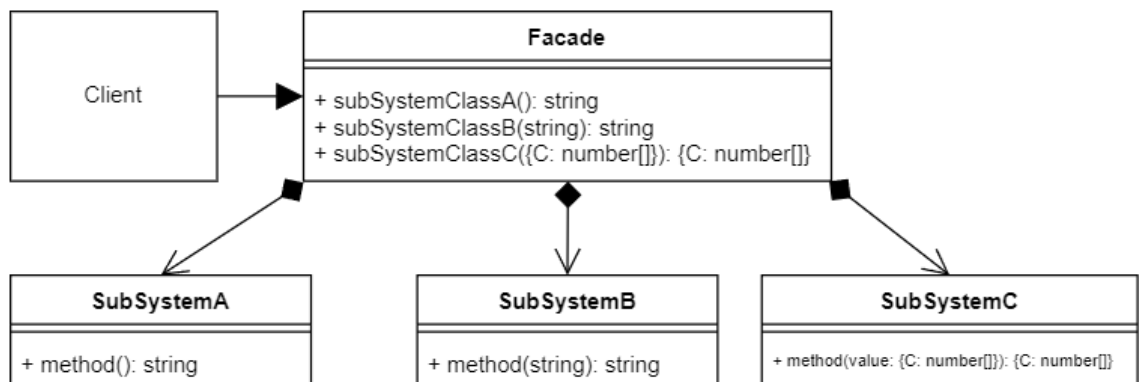
Шаблон може бути застосований якщо, наприклад, реалізацію необхідно змінювати під час роботи програми.

Шаблон Facade

Проблема. Як забезпечити уніфікований інтерфейс з набором розрізнених реалізацій або інтерфейсів, наприклад, з підсистемою, якщо небажаним є високе зв'язування з цією підсистемою або реалізація підсистеми може змінитися?

Рішення. Визначити одну точку взаємодії з підсистемою - фасадний об'єкт, що забезпечує загальний інтерфейс з підсистемою і покласти на нього обов'язок по взаємодії з її компонентами. Фасад - це зовнішній об'єкт, що забезпечує єдину точку входу для служб підсистеми. Реалізація інших компонентів підсистеми закрыта і не доступна для зовнішніх компонентів. Фасадний об'єкт забезпечує реалізацію шаблону "Стійкий до змін" з точки зору захисту від змін у реалізації підсистеми. Структура шаблону наведена на мал. 4.

Facade UML Diagram



Мал. 4. Структура шаблону Facade

Учасники шаблону:

1. **Фасад (Facade)** – це клас, який надає спрощений інтерфейс для взаємодії з підсистемою або групою класів. Фасад приховує деталі роботи внутрішніх компонентів та виконує координацію між ними.

2. **Підсистема (Subsystem)** – це набір класів, які реалізують функціональність системи. Ці класи можуть бути складними і мати внутрішню взаємодію, але фасад приховує цю складність від клієнта.

3. **Клієнт (Client)** – це клас або модуль, який використовує фасад для спрощеного доступу до підсистеми.

Завдання

1. Ознайомитись з призначенням та видами шаблонів проектування ПЗ. Вивчити класифікацію шаблонів проектування ПЗ. Знати назви шаблонів, що відносяться до певного класу.
2. Вивчити структурні шаблони проектування ПЗ. Знати загальну характеристику структурних шаблонів та призначення кожного з них.
3. Детально вивчити структурні шаблони проектування **Composite, Decorator, Proxy, Flyweight, Adapter, Bridge, Facade**. Для кожного з них:
 - вивчити шаблон, його призначення, альтернативні назви, випадки, коли його застосування є доцільним та результати такого застосування;
 - знати особливості реалізації шаблону;
 - вільно володіти структурою шаблону, призначенням його класів та відносинами між ними;
 - вміти розпізнавати шаблон в UML діаграмі класів та будувати сирцеві коди Java-класів, що реалізують шаблон.
4. В підготованому проєкті створити програмний пакет work3. В пакеті розробити інтерфейси і класи, що реалізують завдання 1 та 2 (згідно варіанту) з застосуванням одного чи декількох шаблонів (п.3). В класах, що розробляються, повністю реалізувати методи, пов'язані з функціонуванням Шаблону. Методи, що реалізують бізнес-логіку, закрити заглушками з виводом на консоль інформації про викликаний метод та його аргументи.

Приклад реалізації бізнес-методу:

```
void draw(int x, int y) { System.out.println("Метод draw з параметрами x="+x+" y="+y); }
```

5. За допомогою автоматизованих засобів виконати повне документування розроблених класів (також методів і полів), при цьому документація має в достатній мірі висвітлювати роль певного класу в загальній структурі Шаблону та особливості конкретної реалізації.

Варіанти до завдання 1 (2 бали)

Номер варіанту завдання обчислюється як залишок від ділення номеру залікової книжки на 12.

ВАРІАНТ 0. Клас **GraphicPrimitive**, який зберігає атрибути розміщення графічного примітиву: позиція (x, y), розмір (width, height).

Необхідно розширити функціональність графічних примітивів так, щоб:

- Можна було створювати базові примітиви (наприклад, прямокутник, коло, лінія) з фіксованими параметрами (координати і розмір задаються в конструкторі).

- Можна було створювати композиції примітивів (групи), у яких атрибути розміщення (позиція і розмір) обчислюються на основі вкладених примітивів.
 - Наприклад, позиція групи визначається як мінімальні x та y у серед усіх елементів.
 - Розмір групи – це ширина та висота, які охоплюють усі вкладені примітиви.
- Можна було отримати інформацію про розміщення як для окремих примітивів, так і для композицій.

Вимоги

- Не змінювати існуючий клас *GraphicPrimitive*. Реалізувати можливість гнучкого додавання нових типів примітивів у майбутньому.
- Продемонструвати у методі *main* кілька варіантів роботи з примітивами:
 - окремий прямокутник,
 - окреме коло,
 - група з кількох примітивів,
 - група з вкладеними групами.

ВАРІАНТ 1. Клас *ExpressionNode*, який представляє вузол дерева розбору виразу.

Необхідно розширити функціональність так, щоб:

- Можна було створювати простий вираз: Константа (число), Змінна (ім'я)
- Можна було створювати складний вираз у вигляді дерева: вираз у лапках, що складається з лівого та правого підвиразу і операції (+, -, *, /).
- Можна було отримати структуроване представлення виразу:
 - Для простих виразів – повертається значення або ім'я змінної.
 - Для складних виразів – структура дерева обчислюється, включаючи вкладені вирази.

Вимоги

- Не змінювати існуючий клас *ExpressionNode*. Реалізувати можливість гнучкого додавання нових типів вузлів у майбутньому (наприклад, функцій, унарних операторів).
- Продемонструвати у методі *main* кілька варіантів роботи з деревом:
 - простий вираз-константа,
 - простий вираз-змінна,
 - складний вираз з двома підвиразами,
 - складний вираз з вкладеними складними виразами.

ВАРІАНТ 2. Клас *GameElement*, який представляє елемент ігрового простору.

Необхідно розширити функціональність так, щоб:

- Можна було створювати простий елемент (наприклад, одиночний об'єкт на рівні). Кожний елемент має розмір (ширина та висота) у умовних одиницях.
- Можна було створювати композитний елемент (групу елементів), у якому:
 - Атрибути розміру та площа обчислюються динамічно на основі вкладених елементів.
 - Можливі вкладені композитні елементи (багаторівнева ієрархія).
- Можна було отримати площу елемента:
 - Для простого елемента – $\text{площа} = \text{ширина} \times \text{висота}$.
 - Для композитного елемента – $\text{площа} = \text{сума площ усіх вкладених елементів або мінімальна обгортка, що охоплює всі піделементи (залежно від реалізації)}$.

Вимоги

- Не змінювати існуючий клас *GameElement*. Реалізувати можливість гнучкого додавання нових типів елементів у майбутньому.
- Продемонструвати у методі *main* кілька варіантів роботи з елементами:

- окремий простий елемент,
- група з кількох простих елементів,
- група з вкладеними групами (багаторівнева структура).

ВАРІАНТ 3. Клас **FileSystemItem**, який представляє елемент файлової системи. Необхідно розширити функціональність так, щоб:

- Можна було створювати файл – елементарний об'єкт файлової системи, що має: ім'я файлу, розмір у байтах або умовних одиницях.
- Можна було створювати папку, яка може містити: файли, інші папки (створюючи багаторівневу ієрархію).
- Можна було отримати інформацію про розмір або структуру папки:
 - для файлу – повертається його власний розмір,
 - для папки – розмір обчислюється динамічно як сума розмірів усіх вкладених елементів.

Вимоги

- Не змінювати існуючий клас **FileSystemItem**. Реалізувати можливість гнучкого додавання нових типів елементів у майбутньому (наприклад, спеціальні типи файлів).
- Продемонструвати у методі `main` кілька варіантів роботи з файловою системою:
 - окремий файл,
 - папка з кількома файлами,
 - папка з вкладеними папками (багаторівнева структура).

ВАРІАНТ 4. Клас **Employee**, який представляє співробітника організації.

Необхідно реалізувати функціональність так, щоб:

- Можна було створювати окремого співробітника, який має: ім'я, посаду, зарплату.
- Можна було створювати групу співробітників (відділ), яка може містити: окремих співробітників, інші підгрупи (створюючи багаторівневу ієрархію).
- Метод `displayInfo()` повинен працювати як для окремого співробітника, так і для групи:
 - для співробітника – виводить ім'я, посаду та зарплату,
 - для групи – виводить сумарну інформацію про всіх співробітників та підгрупи, включаючи вкладені групи.

Вимоги

- Не змінювати існуючий клас **Employee**. Реалізувати можливість гнучкого додавання нових типів співробітників або підгруп у майбутньому.
- Продемонструвати у методі `main` кілька варіантів роботи з організаційною структурою:
 - окремий співробітник,
 - група співробітників,
 - група з вкладеними підгрупами (багаторівнева структура).

ВАРІАНТ 5. Клас **GraphicElement**, який представляє базовий графічний елемент у векторному редакторі.

Необхідно реалізувати функціональність так, щоб:

- Можна було створювати простий графічний елемент, наприклад: коло, прямокутник, лінію тощо, кожен елемент має базові атрибути (колір, розмір тощо).
- Можна було динамічно додавати додаткові атрибути відображення: рамку (з товщиною та кольором), тінь, інші ефекти у майбутньому.
- Метод `draw()` повинен відображати елементи у певному порядку (наприклад, тінь → рамка → базовий елемент).

- Метод `getDescription()` повинен повертати текстовий опис елемента, включаючи всі застосовані ефекти. Наприклад: "Коло червоного кольору з рамкою товщиною 2 пікселі і тінню".

Вимоги

- Не змінювати існуючий клас `GraphicElement`. Реалізувати можливість гнучкого додавання нових ефектів у майбутньому.
- Продемонструвати у методі `main` кілька варіантів роботи з елементом:
 - базове коло,
 - коло з рамкою,
 - коло з тінню,
 - комбіноване коло (рамка + тінь).

ВАРІАНТ 6. Клас `GraphicElement`, який представляє базовий графічний елемент у векторному редакторі.

Необхідно реалізувати функціональність так, щоб:

- Можна було створювати прості графічні елементи, наприклад: коло, прямокутник, лінію тощо; кожен елемент має базові атрибути (колір, розмір, положення).
- Можна було додавати додаткові графічні ефекти у вигляді обгортки: кольоровий фон, градієнт, текстура, інші декоративні ефекти у майбутньому.
- Метод `draw()` повинен відображати елемент із застосованими ефектами у правильному порядку.
- Метод `getDescription()` повинен повертати текстовий опис елемента та всіх застосованих ефектів. Наприклад: "Прямокутник синього кольору з градієнтом і текстурою"

Вимоги

- Не змінювати існуючий клас `GraphicElement`. Реалізувати можливість гнучкого додавання нових ефектів у майбутньому.
- Продемонструвати у методі `main` кілька варіантів роботи з елементом:
 - базовий елемент без ефектів,
 - елемент із одним ефектом (наприклад, фон),
 - елемент із кількома ефектами (градієнт + текстура).

ВАРІАНТ 7. Клас `GraphicElement`, який представляє базовий графічний елемент у векторному редакторі.

Необхідно реалізувати функціональність так, щоб:

- Можна було створювати маніпулятори, які дозволяють змінювати геометричні властивості елемента: позицію (координати x, y), розмір (ширина та висота).
- Маніпулятори повинні діяти як об'єкти, що обгортають базовий елемент, забезпечуючи зміну властивостей без модифікації самого базового класу.
- Метод `apply()` або аналогічний повинен виконувати зміну властивостей елемента.
- Метод `getDescription()` повинен повертати текстовий опис елемента після застосування маніпулятора, включаючи нові геометричні параметри.

Вимоги

- Не змінювати існуючий клас `GraphicElement`. Реалізувати можливість гнучкого додавання нових типів маніпуляторів у майбутньому (наприклад, обертання, масштабування, нахил).
- Продемонструвати у методі `main` кілька варіантів роботи з елементом:
 - зміна позиції елемента,
 - зміна розміру елемента,
 - комбіноване застосування кількох маніпуляторів.

ВАРІАНТ 8. Клас **TextElement**, який представляє базовий текстовий елемент у редакторі.

Необхідно реалізувати функціональність так, щоб:

- Можна було створювати прості текстові елементи, які зберігають текст.
- Можна було динамічно змінювати відображення тексту за допомогою різних обгорт: приведення тексту до верхнього регістру, приведення тексту до нижнього регістру, додавання символу нової строки в кінець тексту, інші ефекти у майбутньому.
- Метод `display()` або аналогічний повинен відображати текст із застосованими ефектами.
- Метод `getDescription()` повинен повертати текстовий опис елемента та всіх застосованих ефектів. Наприклад: "Текстовий елемент у верхньому регістрі з новою строкою в кінці".

Вимоги

- Не змінювати існуючий клас **TextElement**. Реалізувати можливість гнучкого додавання нових обробок тексту у майбутньому.
- Продемонструвати у методі `main` кілька варіантів роботи з елементом:
 - базовий текст без ефектів,
 - текст у верхньому регістрі,
 - текст з додаванням нової строки,
 - комбінований текст (верхній регістр + нова строка).

ВАРІАНТ 9. Клас **Message**, який представляє базове текстове повідомлення в чаті.

Необхідно реалізувати функціональність так, щоб:

- Можна було створювати звичайні текстові повідомлення.
- Можна було динамічно додавати різні способи обробки повідомлень, наприклад: форматування у HTML (обгортання тегоми `<html><body>...</body></html>`), шифрування (наприклад, обертання тексту у зворотному порядку), інші види обробки у майбутньому.
- Метод `getContent()` повинен повертати текст повідомлення з урахуванням застосованих обробок.
- Повинна бути можливість комбінувати кілька обробок (наприклад, спочатку шифрування, потім HTML-оформлення).

Вимоги

- Не змінювати існуючий клас **Message**. Реалізувати можливість гнучкого додавання нових обробок повідомлень у майбутньому.
- Продемонструвати у методі `main` кілька варіантів роботи з повідомленням:
 - звичайне повідомлення,
 - повідомлення з HTML-оформленням,
 - зашифроване повідомлення,
 - комбіноване повідомлення (HTML + шифрування).

ВАРІАНТ 10. Клас **Image**, який представляє базове зображення.

Необхідно реалізувати функціональність так, щоб:

- Можна було створювати великі зображення, які можуть мати високі вимоги до пам'яті.
- Забезпечити механізм кешування, щоб: уникнути повторного завантаження даних з диску чи мережі, зменшити споживання пам'яті, зберігати вже завантажене зображення у проміжному сховищі.

- Реалізувати метод `getPixelColor(x, y)` або аналогічний, який повертає колір точки за координатами (x, y) на зображенні.
- Система повинна динамічно завантажувати та обробляти зображення лише при необхідності, забезпечуючи ефективне використання ресурсів.

Вимоги

- Не змінювати існуючий клас *Image*. Реалізувати можливість гнучкого додавання нових способів обробки зображень у майбутньому (фільтри, трансформації, ефекти).
- Продемонструвати у методі `main` кілька варіантів роботи:
 - завантаження зображення та отримання кольору точки,
 - повторне звернення до того ж зображення з використанням кешу,
 - комбіноване використання кешування та обробки зображення.

ВАРІАНТ 11. Клас *Image*, який представляє базове зображення.

Необхідно реалізувати функціональність так, щоб:

- Можна було створювати зображення з контролем доступу до окремих точок.
- Доступ до точок (x, y) повинен бути дозволений лише якщо координати потрапляють у визначений діапазон: $x1 < x < x2$, $y1 < y < y2$, значення $x1$, $x2$, $y1$, $y2$ задаються при створенні об'єкта.
- Реалізувати метод `getPixelColor(x, y)`, який повертає колір точки тільки якщо доступ дозволений; в іншому випадку – повідомляє про відсутність доступу.
- Система повинна дозволяти захищене маніпулювання зображенням, залишаючи внутрішні дані недоступними за межами дозволеної області.

Вимоги

- Не змінювати існуючий клас *Image*.
- Реалізувати можливість гнучкого додавання нових правил контролю доступу у майбутньому.
- Продемонструвати у методі `main` кілька варіантів роботи:
 - доступ до точок у межах дозволеної області,
 - спроба доступу до точок за межами дозволеної області,
 - комбіноване використання доступу та обробки зображення.

Варіанти до завдання 2 (2 бали)

Номер варіанту завдання обчислюється як залишок від ділення номеру залікової книжки на 11.

0. Визначити специфікації класів та реалізацію методів для об'єктів-гліфів латинського алфавіту та об'єктів-рядків. Об'єкти-гліфи мають атрибуту координат та використовуються в якості складових при побудові композитних об'єктів-рядків. Забезпечити ефективне використання пам'яті при роботі з великою кількістю об'єктів-гліфів. Реалізувати метод виводу рядка.
1. Визначити специфікації класів, які подають графічні об'єкти у редакторі растрової графіки – примітиви (точка) та їх композиції (прямокутне зображення). Забезпечити ефективне використання пам'яті при роботі з великою кількістю графічних об'єктів. Реалізувати метод рисування графічного об'єкту.
2. Визначити специфікації класів, які подають графічні об'єкти у редакторі векторної графіки - примітиви (лінія) та їх композиції (прямокутник, трикутник). Забезпечити ефективне використання пам'яті при роботі з великою кількістю графічних об'єктів. Реалізувати метод малювання графічного об'єкту.

3. Визначити специфікації класів, які подають об'єкти-іконки для зображення елементів файлової системи при побудові графічного інтерфейсу користувача (GUI) – примітиви (файли) та їх композиції (директорії). Забезпечити ефективне використання пам'яті при роботі з великою кількістю графічних об'єктів. Реалізувати метод рисування графічного об'єкту.
4. Визначити специфікації класів, які подають графічні об'єкти у редакторі векторної графіки - примітиви (точка) та їх композиції (коло). Примітиви мають атрибути координат в декартовій системі, а об'єкти-композиції — в полярній. Відповідно інтерфейс точки містить методи `setX(int)` та `setY(int)`, а метод малювання кола може оперувати лише методами `setRo(double)`, `setPhi(double)` примітива (які відсутні в класі точки). Забезпечити можливість використання функціональності точки при малюванні кола без зміни інтерфейсу точки та методу малювання кола.
5. Визначити специфікації класів, які подають графічні об'єкти у редакторі векторної графіки - примітиви (точка) та їх композиції (лінія). Примітиви мають цілочисельні атрибути координат (в пікселях), а об'єкти-композиції — дійсні (в сантиметрах). Відповідно інтерфейс точки містить методи `setX(int)` та `setY(int)`, а метод малювання лінії може оперувати лише методами `setX(double)`, `setY(double)` примітива (які відсутні в класі точки). Забезпечити можливість використання функціональності точки при малюванні лінії без зміни інтерфейсу точки та методу малювання лінії.
6. Визначити специфікації класів, які подають графічні об'єкти у редакторі векторної графіки – примітиви (лінія) та їх композиції (прямокутник). Примітиви мають атрибути координат в системі з центром координат в лівому верхньому куті екрана з інверсною віссю абсцис, а об'єкти-композиції — з центром координат посередині екрана і стандартним напрямком осі абсцис. Забезпечити можливість використання функціональності лінії при малюванні прямокутника без зміни методів точки та методу малювання лінії.
7. Визначити специфікації класів, які подають елементи графічного інтерфейсу користувача (GUI) — кнопка, вікно, тощо. Забезпечити розділення абстракції і реалізації таким чином, щоб елементи інтерфейсу могли мати реалізації для різних бібліотек (наприклад, `Qt` та `GTK`) прозорі для користувача. Реалізувати метод малювання елемента.
8. Визначити специфікації класів, які подають графічні об'єкти у редакторі векторної графіки (прямокутник) через різні інтерфейси `API1` та `API2`. Забезпечити прозору для користувача можливість заміни реалізації графічних об'єктів. Реалізувати метод малювання елемента.
9. Визначити специфікації класів, які подають об'єкти для маніпулювання елементами файлової системи - файлами та директоріями. Інтерфейс файлу містить методи `open(String path, boolean createIfNotExist)`, `close()` та `delete(String path)` для відкриття, закриття та видалення файлу (при `createIfNotExist=true` файл буде створений, якщо він не існує або обрізаний до нульової довжини, якщо існує). Інтерфейс директорії містить методи `create(String path)`, та `rmdir(String path)` для створення та видалення директорії. Задати підсистему з 3-ох файлів та 2-х директорій. Забезпечити можливість створення та видалення такої підсистеми через методи `create()`, `destroy()` та зміни структури підсистеми без впливу на її користувача.
10. Визначити специфікації класів, які подають графічні об'єкти у редакторі векторної графіки — лінія (`Line`) та растрове зображення (`Image`). Інтерфейс лінії містить метод `setOpacity(double op)`, що регулює рівень її непрозорості між повністю непрозорою (`op == 1.0`) та невидимою (`op == 0.0`). Інтерфейс растрового зображення

містить метод `setTransparency (double tr)`, що регулює рівень її прозорості між повністю прозорою (`tr == 1.0`) та повністю непрозорою (`tr == 0.0`). Задати підсистему з зображення та ліній, які його обрамляють. Забезпечити можливість вмикання/вимикання відображення підсистеми через метод `show(boolean vis)`, та зміни типу обрамлення (горизонтальне, вертикальне, повне, тощо) без впливу на користувача такої підсистеми.

Питання для самостійної перевірки

1. Шаблони проектування програмного забезпечення. Призначення.
2. Класифікація шаблонів проектування ПЗ.
3. Призначення структурних шаблонів проектування ПЗ.
4. Коротка характеристика кожного структурного шаблону.
5. Призначення шаблону Composite.
6. Структура шаблону Composite та його учасники.
7. Особливості реалізації шаблону Composite. Результат використання шаблону.
8. Призначення шаблону Decorator.
9. Структура шаблону Decorator та його учасники.
10. Особливості реалізації шаблону Decorator. Результат використання шаблону.
11. Призначення шаблону Proxy.
12. Структура шаблону Proxy та його учасники.
13. Особливості реалізації шаблону Proxy. Результат використання шаблону.
14. Назви, призначення та мотивація шаблону Flyweight.
15. Структура шаблону Flyweight та його учасники.
16. Особливості реалізації шаблону Flyweight. Результат використання шаблону.
17. Назви, призначення та мотивація шаблону Adapter.
18. Структура шаблону Adapter та його учасники.
19. Особливості реалізації шаблону Adapter. Результат використання шаблону.
20. Назви, призначення та мотивація шаблону Bridge.
21. Структура шаблону Bridge та його учасники.
22. Особливості реалізації шаблону Bridge. Результат використання шаблону.
23. Назви, призначення та мотивація шаблону Facade.
24. Структура шаблону Facade та його учасники.
25. Особливості реалізації шаблону Facade. Результат використання шаблону.
26. Відмінність Adapter, Decorator та Proxy в специфікації конструктора.
27. Шаблони, які використовуються сумісно з Flyweight, Adapter, Bridge, Facade.

Протокол

Протокол має містити титульну сторінку (з номером залікової книжки), завдання, роздруківку діаграми класів, розроблений програмний код та генеровану документацію в форматі JavaDoc. Файли вихідного коду (*.java) повинні бути додані до протоколу.

Список рекомендованих інформаційних джерел

1. Design Patterns: elements of reusable object-oriented software / Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. Indianapolis: - Addison-Wesley, 1994. - 417 p. ISBN: 0201633612.
2. Mark Grand. Patterns in Java: A Catalog of Reusable Design Patterns Illustrated with UML, Volume 1, 2nd Edition. - Wiley Publishing, 2002. - 480 p. ISBN: 0471258393
3. Design Patterns. URL: https://en.wikipedia.org/wiki/Design_Patterns
4. Design Pattern – Overview. URL:
https://www.tutorialspoint.com/design_pattern/design_pattern_overview.htm
5. Structural Design Patterns. URL: <https://refactoring.guru/design-patterns/structural-patterns>
6. Structural Design Pattern. URL: <https://www.scaler.com/topics/design-patterns/structural-designpattern/>
7. What is Structural Design Pattern? URL:
<https://www.scaler.com/topics/designpatterns/structural-design-pattern/>
8. Structural design patterns. URL: <https://www.javatpoint.com/structural-design-patterns>
9. Structural pattern – Wikipedia. URL: https://en.wikipedia.org/wiki/Structural_pattern
10. Structural patterns. URL: https://sourcemaking.com/design_patterns/structural_patterns/